

Project Report



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Performance Analysis and Modeling of Software Systems (PAMS) - Summer Semester 2026

Dang Minh Nguyen
Matriculation Number: 2809458
May 13, 2026

Contents

1	Application Description and Design	2
1.1	General Description	2
1.2	Detailed Description of Modules Under Test	2
2	Benchmark Harness Design / Application Changes	3
2.1	Experimental Design and Phased Execution	4
2.2	Code Changes for Logging (and Benchmark Client)	4
2.3	Scripts for Running Experiments	4
2.4	Data Collection	5
2.5	Data Processing and Statistics	5
2.6	Plotting Method	5
3	Experiments	5
3.1	Environment	5
3.2	Experiment 1: Cache Effectiveness	6
3.2.1	Setup	6
3.2.2	Hypothesis	6
3.2.3	Results and Discussion	7
3.3	Experiment 2: Cache Size and Eviction Policy	7
3.3.1	Setup	7
3.3.2	Hypothesis	7
3.3.3	Results and Discussion	7
3.4	Experiment 3: Increasing number of workers	7
3.4.1	Setup	8
3.4.2	Hypothesis	8
3.4.3	Results and Discussion	8
4	Modeling of the Application	8

Guidelines and Requirements (Please Delete!)

Formatting You must hand in a single PDF document as the project report, with your Matriculation Number clearly visible on the title page. The total length of the final PDF may not exceed 20 pages (including all sections).

We recommend that you use this L^AT_EX template to write your report. This is, however, not a requirement, and you can use Word or a different L^AT_EX template as long as you use an A4 page size with at least 1cm margins and a font size of 11pts or larger.

Structure Writing your report structured according to the Sections and Subsections defined in this document is required. Feel free, however, to organize your descriptions, experiments, graphs, and discussions as you see fit within this high-level structure. Depending on what you've implemented as part of the project and what additional explorations you undertook, one of the sections may be significantly longer than the others. This is fine, but for a good grade, you must ensure that all tasks/sections are well covered in your report!

If you use this template for your report, you should begin by removing this "Guidelines and Requirements" section and all explanatory text from within the sections before you start writing. We included some examples of adding figures to the document, so feel free to remove them as well.

Grading You can achieve 100 pts in total, whereby each section is equally important: 25 pts for *Application Description and Design*, 25 pts for *Benchmark Harness Design / Application Changes*, 30 pts for *Experiments* (10 pts for each *Experiment*), and 20 pts for *Modeling of the Application*.

VM virtual machine

MVCC multi-version concurrency control

1 Application Description and Design

ToyDB [1] is a distributed SQL database written in Rust and developed primarily as an educational project. Its goal is not to compete with production systems in terms of performance or scalability that demonstrates how a modern distributed database is structured and how its components interact.

- Raft-based distributed consensus for linearizable state-machine replication.
- ACID transactions with multi-version concurrency control (MVCC) and snapshot isolation.
- A pluggable storage engine with BitCask-style and in-memory backends.
- An iterator-based query engine with simple heuristic optimization and time-travel support.
- A SQL interface supporting basic DDL/DML, joins, aggregates, and transactions.

toyDB was originally written around 2020 as a vehicle to explore database internals and to serve as a didactic counterpart to large-scale distributed SQL systems such as CockroachDB [2].

1.1 General Description

In this project, I use toyDB as the database system and treat it as an example of a distributed relational DBMS.

ToyDB follows the architecture of a distributed database. Clients communicate with a toyDB node using a SQL interface. Incoming SQL statements are parsed and then transformed into a logical query plan using storage in engine and finally into a physical execution plan. By default, toyDB cluster has 5 nodes.

This project builds on toyDB's existing architecture and uses its main components.

Instead of designing again the system from scratch, I keep the core components (Raft-based replication, MVCC transactions, and the iterator-based query engine) intact and explore targeted extensions and modifications. Subsequent sections I will introduce a single optimization idea: introducing a caching frontend for toyDB.

1.2 Detailed Description of Modules Under Test

In this project, I would like to implement a **cache on frontend for toyDB** and benchmark the its performance on **Google Cloud**.

The cache stores the response for different requests and if missing refers to the DB. Two replacement strategies, LRU and FIFO, are used in the experiments to see how the frontend cache influences database performance under different key ranges, different strategies, and different numbers of workers.

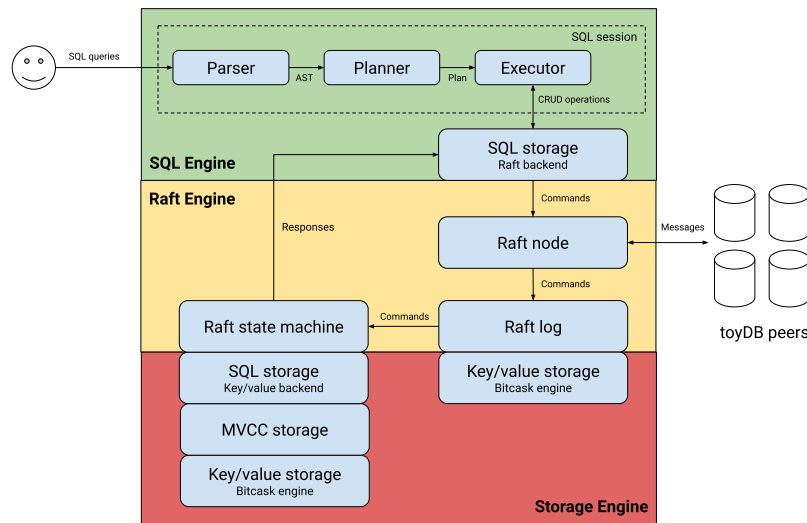


Figure 1: Architecture of toyDB [1]

2 Benchmark Harness Design / Application Changes

The benchmark client ([workload.rs](#)) can connect to the cache frontend by overriding the host list using `-H`, without any changes to the client logic. On each request, the frontend checks an in-memory cache. On a cache hit, it returns the cached result directly. On a miss, it forwards the request to the backend toyDB cluster, caches the response (subject to eviction policy), and returns it to the client.

To evaluate the performance impact of the caching frontend, I will reuse toyDB's existing workload benchmark client ([src/bin/workload.rs](#)). The workload tool spawns a configurable number of concurrent worker threads (`-concurrency`) and measures end-to-end transaction latency on the client side using `Instant::now()` and an HDR histogram (`hdrhistogram`) to report throughput and latency percentiles (`p50/p90/p99/pMax`).

Latency percentiles (`p50/p90/p99/pMax`). In this project, the benchmark is evaluated using a set of metrics `p50`, `p90`, `p99`, `pMax`

- `p50` is the median latency: 50% of requests finish within the reported latency.
- `p90` is the 90th percentiles: 90% of requests finish within the reported latency.
- `p99` is the 99th percentiles: 99% of requests finish within the reported latency.
- `pMax` is the maximum latency observed so far in the run (worst single transaction).

These higher percentiles characterize *tail latency*, which is often critical for user-perceived performance and can increase under contention, queuing, or outliers.

Without the cache, the workload client connects directly to the toyDB cluster nodes (default hosts `localhost:9601..9605` as started by `cluster/run.sh`). For the cache configuration, the same workload client connects to the cache frontend by overriding the host list via `-H`, thereby keeping the workload generator and measurement methodology identical. This enables an A/B comparison by only changing the target hosts.

The benchmark is run as a **closed system** in the sense of the lectures: I use a fixed number of client threads, and each thread waits for the response before sending the next request.

For the cache experiments, I focus on two main factors: the size of the dataset and the cache eviction policy (LRU vs FIFO).

In this project, 2 different datasets are used to benchmark the performance:

- a **small dataset** (e.g., `read -rows 10000`), and
- a **large dataset** (e.g., `read -rows 10000000`).

I expect higher cache hit ratios primarily under workloads with temporal locality (e.g., repeated access to a hot subset of keys); dataset size is used here to stress capacity effects and to amplify the performance difference between hits and misses.

For each dataset size, I will run the benchmark with and without the caching frontend and compare throughput and latency percentiles (`p50/p90/p99/pMax`).

2.1 Experimental Design and Phased Execution

Rather than implementing a separate cache pre-loading routine, the benchmark prepares the initial dataset via the workload's `prepare()` step. During execution, the workload prints per-second statistics. For analysis, I treat the first few seconds as warm-up and focus on the subsequent steady-state interval when comparing configurations.

Experiment	Title	Core Objective
Exp 1	Cache Effectiveness	Quantify how adding a frontend cache changes throughput and latency percentiles, and how this improvement depends on dataset size, and the resulting hit/miss ratio.
Exp 2	LRU and FIFO on different cache size	Compare between LRU and FIFO on different key ranges and cache sizes. Key range can also be bigger than the cache size.
Exp 3	Increasing number of workers	Increase the number of workers with different concurrency levels $K \in \{4, 6, 8\}$ to observe how the cache and the database perform under increasing load. Use a suitable cache capacity (and policy) selected based on the results of Experiment 2.

Table 1: Summary of Experimental Framework

2.2 Code Changes for Logging (and Benchmark Client)

In the benchmark client ([workload.rs](#)), each worker measures request latency using `Instant`. Every second, the benchmark prints throughput (txns/s) and latency percentiles (p50/p90/p99/pMax). This ensures the measured metrics are end-to-end and comparable across configurations.

To additionally explain cache behavior, I will add counters in the cache frontend (hits, misses, evictions) and report hit ratio alongside the workload's throughput/latency results.

I also extended the benchmark client with a new query type for range selections, called **Range**. While the original `read` workload issues point lookups on a single key, the `Range` variant generates queries that retrieve all rows in a key interval.

For the experiments, I updated the benchmark client so that each benchmark run automatically produces results in CSV files without requiring manual file paths. Specifically, I added two runner-level CLI options:

- `-experiment`: a human-readable experiment tag used in output filenames.
- `-out-dir`: output directory for benchmark artifacts (defaults to `csv/`).
- At startup, the client ensures the output directory exists via `create_dir_all(out_dir)` and generates a unique `run_id` from `SystemTime` (milliseconds since `UNIX_EPOCH`).
- It then writes per-second measurements to `<out-dir>/<experiment>-<run_id>.csv` and writes the final one-row summary to `<out-dir>/<experiment>-<run_id>-summary.csv`.

Otherwise, I split the original [workload.rs](#) into `bank.rs`, `range.rs`, `read.rs`, `write.rs`. Details can be found at [my adjustment code](#).

2.3 Scripts for Running Experiments

I plan to use a small set of scripts that consistently (i) start the system under test, (ii) execute the benchmark client with a predefined parameter set, and (iii) collect the produced CSV logs for later analysis.

Benchmark client (CSV instrumentation). The benchmark runner in [workload.rs](#) was modified to export results automatically as CSV files. Each run is tagged with an `-experiment` name and a unique `run_id` (timestamp) to avoid overwriting previous results. Output is written to `-out-dir` (default: `csv/`).

The runner exports two csv files per run:

- **Per-second time series CSV** (one row per second) with columns `time_s`, `progress`, `txns`, `rate_tps`, `p50_ms`, `p90_ms`, `p99_ms`, `pmax_ms`.
- **Final summary CSV** (one row per run) with columns `experiment`, `run_id`, `workload`, `hosts`, `concurrency`, `count`, `seed`, `total_time_s`, `txns`, `rate_tps`, `p50_ms`, `p90_ms`, `p99_ms`, `pmax_ms`.

Experiment driver scripts (Exp1–Exp3). I organize automation as one script per experiment, where each script runs a small *matrix* of configurations and repetitions and stores outputs under consistent experiment tags. Concretely, each script: (i) starts or verifies the cluster is running, (ii) iterates over the factor levels relevant to the experiment, (iii) runs a fixed number of repetitions (e.g., 3–5) with controlled seeds, and (iv) writes all CSV artifacts to `csv/` using descriptive `–experiment` names.

- **Exp1 (Cache effectiveness):** one script runs baseline vs. cache for the chosen dataset sizes and locality settings, keeping workload type and client settings fixed to isolate cache impact.
- **Exp2 (Eviction policy and cache size):** one script runs the same workload for multiple cache sizes and eviction policies (LRU vs FIFO), sweeping key ranges and locality levels while holding concurrency and dataset size constant.
- **Exp3 (increase number of clients):** one script implements the factorial plan by enumerating all combinations of factors (e.g., cache on/off and low/high client concurrency). Each combination is executed with the same repetition protocol and logged with a systematic experiment tag so results can be grouped and compared during analysis.

This structure keeps scripts short and purpose-specific: each experiment has a single entry point, and all runs follow the same conventions for naming, repetition, and CSV output.

2.4 Data Collection

All benchmark results are written to the `csv/` directory as CSV files generated by the benchmark client. I extended the default workload runner in [workload.rs](#) to automatically export measurements for each run.

Each benchmark execution produces two files:

- a **per-second time series** CSV containing `time_s`, `progress`, `txns`, `rate_tps`, `p50_ms`, `p90_ms`, `p99_ms`, `pmax_ms`, and
- a **final one-row summary** CSV containing the run configuration (e.g., `hosts`, `concurrency`, `count`, `seed`) and the final throughput and latency percentiles.

Filenames include the `–experiment` tag and a unique `run_id` timestamp to prevent overwriting and to enable grouping runs by configuration.

Benchmark runs are executed via experiment driver scripts (one per experiment), which standardize the parameter sets (workload type, dataset size, concurrency, seed, and whether the cache is enabled and which eviction policy is used) and thus ensure consistent data collection across repetitions. After each run, the generated CSV files are collected directly from `csv/` and used as input for subsequent data processing and plotting.

2.5 Data Processing and Statistics

For all experiments I discard the first 5 s of each run as warm-up and compute mean and sample standard deviation over the remaining steady-state interval.

2.6 Plotting Method

The benchmark client produces two CSV files per run: a per-second time series and a final one-row summary (cf. Section 2). For each experiment, I process these CSV files as follows:

- **Parsing and pruning:** I visualize the CSVs using **matplotlib** and **seaborn**.

3 Experiments

3.1 Environment

For all experiments, I separate each node on a different virtual machine ([VM](#)) on Google Cloud.

3.2 Experiment 1: Cache Effectiveness

Experiment 1: Cache Effectiveness: This experiment measures the impact of dataset size and eviction strategies on throughput and latency percentiles. By comparing small (e.g., `-rows 10000`) and large (e.g., `-rows 10000000`) datasets, I identify the performance gains provided by memory-resident data versus the latency penalties of backend fetches. I treat the first few seconds as warm-up and focus on the steady-state interval when comparing configurations.

Purpose: The goal of this experiment is to quantify how an in-memory caching frontend changes toyDB's throughput and latency. By comparing small and large datasets, with and without the cache, I aim to understand how cache hit ratio and dataset size influence performance.

3.2.1 Setup

I plan to run this experiment on Cloud using five VM in the same region. All machines run with same OS and Rust version.

Server nodes (toyDB cluster). Each VM hosts one toyDB node, forming a five-node cluster:

- **n1:** e2-standard-2 (2 vCPUs, 8 GiB RAM), IP 10.0.0.11
- **n2:** e2-standard-2, IP 10.0.0.12
- **n3:** e2-standard-2, IP 10.0.0.13
- **n4:** e2-standard-2, IP 10.0.0.14
- **n5:** e2-standard-2, IP 10.0.0.15

On each VM I run a toydb process listening on port 9601 and configured with the full cluster membership. The nodes use the default BitCask-based storage backend.

Cache frontend and benchmark client. The cache frontend and the benchmark client run on VM n1 alongside the toyDB node:

- The **cache frontend** is a separate process on n1, listening on 127.0.0.1:9701. It is configured with a cache capacity of C entries and uses an LRU eviction policy in this experiment. On a cache miss, it forwards requests to the backend cluster via the internal IPs 10.0.0.11:9601, ..., 10.0.0.15:9601.
- The **benchmark client** is toyDB's workload tool (`src/bin/workload.rs`), also running on n1. It uses a closed-loop configuration with `-concurrency K` worker threads, so each thread waits for the response before issuing the next request.

Workload and configurations. The workload is the built-in read benchmark. The dataset size is controlled via the `-rows` parameter:

- **Small dataset:** `-rows 10000`
- **Large dataset:** `-rows 10000000`

For each dataset size, I evaluate two configurations:

1. **Baseline (no cache):** the client on n1 connects directly to all toyDB nodes using `-H 10.0.0.11:9601,10.0.0.12:9601,10.0.0.13:`
2. **Cache enabled:** the client on n1 connects to the local cache frontend using `-H 127.0.0.1:9701`; the frontend in turn forwards cache misses to the same backend cluster.

Each run lasts T seconds. During this time, the client records per-second throughput and latency percentiles (p50, p90, p99, pMax) to CSV, while the cache frontend counts hits and misses to compute the cache hit ratio.

3.2.2 Hypothesis

- **Cache vs. no cache:** For a fixed workload and dataset size, enabling the cache will increase throughput (txns/s) and decrease latency percentiles (p50, p90, p99) compared to running the workload directly against the toyDB cluster without a cache.
 - **Effect of dataset size:** The relative performance benefit of the cache will be larger for the large dataset (e.g., `-rows 10000000`) than for the small dataset (e.g., `-rows 10000`), because the larger dataset no longer fits into main memory and therefore causes more expensive backend accesses on cache misses.
 - **Hit ratio correlation:** Across all runs, configurations with a higher cache hit ratio will show higher throughput and lower latency percentiles than configurations with a lower hit ratio.
-

3.2.3 Results and Discussion

Describe and discuss the results and how/why they match or don't match the hypothesis, including, e.g., bottleneck analysis, simple performance scaling prediction.

3.3 Experiment 2: Cache Size and Eviction Policy

Experiment 2: Cache Size and Eviction Policy. In this experiment, the efficiency of different eviction strategies (LRU vs FIFO) is evaluated for a range of cache sizes and key ranges. I compare performance across small and large caches, and across workloads with varying degrees of temporal locality, to understand when a more sophisticated eviction policy (LRU) provides a clear benefit over a simpler one (FIFO).

Purpose: The goal of this experiment is to quantify how cache capacity and eviction policy affect throughput, latency, and cache hit ratio. By comparing runs with the same workload but different cache sizes and policies, I aim to identify: (i) the regimes where LRU significantly outperforms FIFO, (ii) where both behave similarly, and (iii) how sensitive the system is to cache sizing errors (too small vs. comfortably large caches).

3.3.1 Setup

I use the same closed-loop benchmark client (`workload.rs`) with `-concurrency K` worker threads and a fixed runtime of T seconds per run. The first 5 seconds are treated as warm-up and excluded from the analysis. The workload is read-heavy and uses the `read` (and, where applicable, `Range`) query types introduced in Section 2.

In contrast to Experiment 1, here I systematically vary three factors:

- **Dataset size:**
 - *Small*: `-rows 10000`
 - *Large*: `-rows 10000000`

This lets me study eviction behavior both when the working set can fit into the cache (small dataset) and when it clearly cannot (large dataset).

- **Cache size:** a *small* and a *large* cache, configured via the cache frontend's `-capacity` parameter. The small cache can hold approximately S_{small} distinct keys (a small fraction of the large dataset), while the large cache can hold S_{large} keys (enough to contain the entire small dataset and a larger fraction of the large one).
- **Eviction policy:** LRU vs. FIFO, selected via a `-policy` flag in the cache frontend.

For each combination (e.g., *small dataset* + *small cache* + *LRU*), I will run the benchmark three times with identical parameters (hosts, concurrency, row count, and random seed pattern). During each run, the client records per-second throughput and latency percentiles (p50, p90, p99, pMax) to CSV, while the cache frontend maintains counters for hits, misses, and evictions. From these logs I compute steady-state averages and standard deviations for throughput, latency, and cache hit ratio for each configuration.

3.3.2 Hypothesis

- **Working set fits in cache:** When the key range (working set) fits entirely into the cache (i.e., cache capacity $\geq$ number of distinct keys accessed), both LRU and FIFO should quickly converge to a high hit ratio. In this case, I expect similar throughput and latency for LRU and FIFO, and both should clearly outperform the configuration without a cache.
 - **Working set larger than cache with temporal locality** When the key range is larger than the cache but the workload exhibits temporal locality (recently accessed keys are likely to be accessed again), I expect LRU to achieve a higher hit ratio than FIFO. As a result, LRU should provide higher throughput and lower latency percentiles than FIFO, despite its slightly higher bookkeeping overhead.
 - **Working set larger than cache with weak locality** When the key range is significantly larger than the cache and access patterns are close to uniform (low temporal locality), I expect LRU and FIFO to achieve similarly low hit ratios. In this regime, the simpler FIFO policy may show slightly better latency than LRU because it incurs less per-operation overhead while offering no worse hit ratio.
-

3.3.3 Results and Discussion

3.4 Experiment 3: Increasing number of workers

Increase the number of workers with different concurrency levels $K \in \{4, 6, 8\}$ to observe how the cache and the database perform under increasing load. Use a suitable cache capacity (and policy) selected based on the results of Experiment 2.

3.4.1 Setup

Use the parameter **-concurrency** to identify number of concurrent threads $K \in \{4, 6, 8\}$ to observe how the cache and the database perform under increasing load.

The experiment is performed with all types of query

- **Dataset size:**
 - *Small*: `-rows 10000`
 - *Large*: `-rows 10000000`
- **Cache size:** a *small* and a *large* cache, configured via the cache frontend's `-capacity` parameter. The small cache can hold approximately S_{small} distinct keys (a small fraction of the large dataset), while the large cache can hold S_{large} keys (enough to contain the entire small dataset and a larger fraction of the large one).
- **Eviction policy:** LRU vs. FIFO, selected via a `-policy` flag in the cache frontend.

3.4.2 Hypothesis

- As the number of concurrent workers K increases, the average response time of the database (without cache hits) will increase, because the system becomes more loaded and more requests have to wait.
- As K increases, the cache will reduce the load on the database by serving repeated requests, so runs with caching enabled will show lower response time and higher throughput than runs without caching.
- **When K becomes 8** for the chosen cache capacity and number of database workers, we expect to see signs of saturation (e.g., response time growing quickly and throughput increasing more slowly or even flattening).

3.4.3 Results and Discussion

4 Modeling of the Application

In this project, I apply the **M/M/m** queueing model. In this model:

- M stands for the arrival process of client queries/transactions.
- M stands for the service process, where the service times of queries are assumed to be exponentially distributed.
- m stands for the number of servers, i.e., the maximum number of queries processed in parallel.

The purpose of using this model is to evaluate toyDB's performance when the number of concurrent clients increases.

The questions I want to answer are: how the frontend cache affects the performance and the speed of queries when clients are running concurrently. In particular, I want to see whether the frontend cache makes the query time faster and reduces the queueing time.

Input parameters: In this project, I apply the **M/M/m** model to a single toyDB node. The parameters are defined as follows:

- λ : mean arrival rate of client queries to one toyDB node (queries per second).

I will log a timestamp for each request arrival on the node. During a steady period of the experiment (e.g., 60 seconds), I count how many requests arrived, denoted by N . Then I estimate the arrival rate as

$$\lambda \approx \frac{N}{T} \quad [\text{queries/second}],$$

where T is the length of the measurement interval in seconds.

- μ : mean service rate of a single server (worker) on that node, measured in completed queries per second. For each query, I record:
 - the time when a worker starts processing it: t_{start}
 - the time when the worker finishes: t_{end}

I then compute the service time for that query as

$$S = t_{\text{end}} - t_{\text{start}},$$

which excludes any waiting time in the queue.

I collect many service times and compute their average $\bar{S}$ (in seconds). The service rate per worker is then estimated as

$$\mu \approx \frac{1}{\bar{S}} \quad [\text{queries/second per worker}].$$

- m : number of servers (server parallelism), i.e., the maximum number of queries processed in parallel. In the experiments, m corresponds to the concurrency level K , which is set using the `-concurrency` parameter.

Outputs and comparison with measurements.

From the M/M/m model, I obtain performance metrics such as the average response time, average queueing time, and throughput. I compare these with the measurements from toyDB using the following plots:

A line chart with these configurations will be applied to visualize the benchmark results for this application model

- The concurrency level m is on the x-axis
- The average response time on the y-axis, again showing both the model predictions and the measured values. Model predictions are calculated using formulas in the take home lectures.

These plots allow me to see how the performance of toyDB changes as the concurrency level increases and to evaluate how well the M/M/m model matches the observed behaviour.

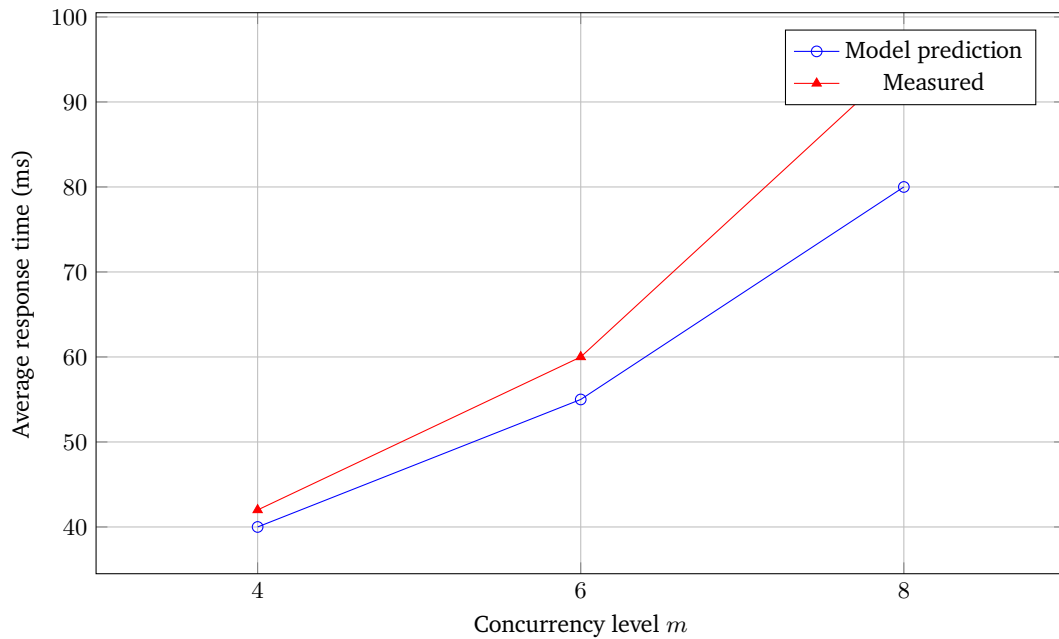


Figure 2: Sample chart: Average query response time as a function of concurrency level m . Model predictions are calculated using formulas from the take-home lectures.

References

- [1] Erik Grinaker. Toydb: A distributed sql database in rust. <https://github.com/erikgrinaker/toydb>, 2020. Accessed: 2026-05-01.
- [2] Rebecca Taft, Irfan Sharif, Andrei Matei, Nathan VanBenschoten, Jordan Lewis, Tobias Grieger, Kai Niemi, Andy Woods, Anne Birzin, Raphael Poss, et al. Cockroachdb: The resilient geo-distributed sql database. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 1493–1509, 2020. Accessed: 2026-05-10.